C Memory Management

malloc, calloc, realloc, free, common bugs and debugging

1. Where variables live

Region	What goes there	Lifetime
Stack	Local variables, function arguments, return addresses	Automatic: freed when the function returns
Heap	Memory you request with malloc / calloc / realloc	Manual: lives until you call free() (or the program ends)
Data / BSS	Global and static variables (BSS = zero-initialized)	Whole program
Text	Your compiled code (and string literals, read-only)	Whole program

Use the heap when the size is unknown at compile time, when the data must outlive the function that created it, or when it is too big for the stack.

```
int  a = 5;                    // stack
int *p = malloc(sizeof(int));  // p itself is on the stack, the int it points to is on the heap
```

2. The basics

- Header: `#include <stdlib.h>`
- All allocators return `void *`, which converts to any pointer type in C. No cast is needed (and a cast is discouraged in C).
- On failure they return `NULL`. **Always check.**
- Sizes are of type `size_t` (unsigned), measured in bytes.
- Every successful allocation must be matched by exactly one `free()`.

3. malloc: allocate raw memory

```
void *malloc(size_t size);
```

Allocates *size* bytes and returns a pointer to the first byte. The contents are **uninitialized** (garbage).

```
int *arr = malloc(4 * sizeof(int));
if (arr == NULL)
{
    perror("malloc");
    return 1;
}
for (int i = 0; i < 4; i++)
    arr[i] = i * 10;       // you must initialize before reading
free(arr);
```

Note: Prefer `malloc(n * sizeof(*arr))`. If you later change the type of arr, the size stays correct automatically.

```
int *arr = malloc(n * sizeof(*arr));   // best practice
```

- **malloc(0)** is implementation-defined: it may return NULL or a unique pointer. Avoid relying on it.
- Watch for overflow in `n * sizeof(*arr)` when n is user-controlled. calloc checks this for you.

4. calloc: allocate zeroed memory

```
void *calloc(size_t count, size_t size);
```

Allocates *count* * *size* bytes and sets **every byte to 0**. It also checks that *count* * *size* does not overflow.

```
int *arr = calloc(4, sizeof(int));   // arr[0..3] are all 0
if (arr == NULL)
    return 1;
```

	malloc	calloc
Arguments	1 (total bytes)	2 (count, element size)
Initial values	Garbage	All bits zero
Overflow check on count*size	No (you must do it)	Yes
Speed	Slightly faster	Slightly slower (must zero)

All-bits-zero means `0` for integers. On virtually every modern system it also means `NULL` for pointers and `0.0` for floating point, but the C standard only guarantees it for integer types.

5. realloc: resize a block

```
void *realloc(void *ptr, size_t new_size);
```

Changes the size of a block that was previously allocated. The existing data is preserved up to the smaller of the old and new sizes. New bytes (when growing) are **uninitialized**.

What can happen

- The block grows or shrinks **in place**: same address returned.
- The block is **moved**: new address returned, old data copied, old block freed automatically. Any old pointers into it are now dangling.
- It **fails**: returns NULL and the **original block is untouched and still valid** (you must still free it).

The classic mistake

```
// WRONG: if realloc fails, arr becomes NULL and the old block leaks
arr = realloc(arr, new_n * sizeof(*arr));
```

The safe pattern

```
int *tmp = realloc(arr, new_n * sizeof(*arr));
if (tmp == NULL)
{
    free(arr);          // original still valid, so clean it up
    return 1;
}
arr = tmp;
```

Special cases

- `realloc(NULL, n)` behaves exactly like `malloc(n)`.
- `realloc(ptr, 0)`: behavior is implementation-defined / deprecated in C23. Use free() instead.
- ptr must come from malloc / calloc / realloc (or be NULL). Never pass stack memory.

Growing array example (dynamic array)

```
size_t cap = 2, len = 0;
```

```
int *v = malloc(cap * sizeof(*v));
if (!v) return 1;

for (int i = 0; i < 10; i++)
{
    if (len == cap)
    {
        cap *= 2;                          // double the capacity
        int *tmp = realloc(v, cap * sizeof(*v));
        if (!tmp) { free(v); return 1; }
        v = tmp;
    }
    v[len++] = i;
}
free(v);
```

6. free: release memory

```
void free(void *ptr);
```

- Returns the block to the allocator so it can be reused. It does **not** change your pointer variable.
- `free(NULL)` is safe and does nothing.
- Only free pointers returned by malloc / calloc / realloc, and only the **exact pointer** you received (not `arr + 1`).
- Free each block **once**. A second free is undefined behavior (double free).
- After freeing, never read or write through the pointer (use after free).

```
free(arr);
arr = NULL;     // good habit: now a double free is harmless and a later use crashes clearly
```

Note: If arr was allocated with calloc(4, sizeof(int)), you still release it with plain free(arr). There is no 'cfree'.

7. Strings and 2D arrays

Strings: remember the +1

```
char *copy = malloc(strlen(src) + 1);   // +1 for the '\0' terminator
if (!copy) return 1;
strcpy(copy, src);
free(copy);

char *dup = strdup(src);   // POSIX shortcut (allocates + copies), also needs free()
```

2D array: array of row pointers

```
int **m = malloc(rows * sizeof(*m));
if (!m) return 1;
for (size_t i = 0; i < rows; i++)
{
    m[i] = calloc(cols, sizeof(**m));
    if (!m[i])
    {
        while (i > 0)              // roll back what was already allocated
            free(m[--i]);
        free(m);
        return 1;
    }
}
/* use m[i][j] */
for (size_t i = 0; i < rows; i++)
```

```
    free(m[i]);                  // free rows first...
free(m);                         // ...then the array of pointers
```

2D array: single contiguous block

```
int *m = calloc(rows * cols, sizeof(*m));   // one malloc, one free
m[i * cols + j] = 42;                       // element (i, j)
free(m);
```

The single-block version is faster, simpler to free, and cache-friendly.

8. Common memory bugs

Bug	Example	Result
Memory leak	malloc without a matching free; losing the only pointer	Memory use grows until exit
Use after free	free(p); p[0] = 1;	Undefined behavior, corruption, security hole
Double free	free(p); free(p);	Heap corruption / crash
Buffer overflow	arr[4] = 1; on a 4-element array	Overwrites neighbouring memory
Uninitialized read	Reading malloc'd memory before writing	Random values
Freeing non-heap	int x; free(&x);	Crash / undefined behavior
Freeing mid-block	free(arr + 1);	Crash / undefined behavior
Wrong size	malloc(sizeof(p)) instead of sizeof(*p)	Allocates pointer size, not struct size
Missing NULL check	Using the result of malloc directly	NULL dereference crash
Dangling pointer	Returning the address of a local variable	Points to dead stack memory

Examples

```
// Wrong size
struct node *n = malloc(sizeof(n));      // BUG: size of a pointer (8 bytes)
struct node *n = malloc(sizeof(*n));     // correct

// Off by one (valid indexes are 0..3)
int *a = malloc(4 * sizeof(*a));
a[4] = 1;                                // BUG: out of bounds

// Dangling pointer
int *bad(void)
{
    int x = 5;
    return &x;                           // BUG: x dies when bad() returns
}

// Leak by overwriting the pointer
char *s = malloc(10);
s = malloc(20);                          // BUG: first block is lost forever
```

9. Best practices checklist

- Always check the return value of malloc / calloc / realloc.
- Use `sizeof(*ptr)` instead of repeating the type.

- Use calloc when you need zeroed memory, or when computing count * size from untrusted input.
- Use a temporary pointer with realloc.
- Set pointers to NULL after free.
- Decide **who owns** each allocation (which function must free it) and document it.
- Free in the reverse order of dependency (inner blocks first, then the container).
- On every early `return`, make sure everything allocated so far is freed. A single 'cleanup' label with `goto` is a common clean pattern.
- Never assume memory is zero unless you used calloc or memset.
- Do not use memory after the function that owns it has freed it.

Cleanup with goto (common C pattern)

```
int run(void)
{
    int  ret = 1;
    char *a = NULL;
    int  *b = NULL;

    a = malloc(100);
    if (!a) goto cleanup;
    b = calloc(10, sizeof(*b));
    if (!b) goto cleanup;

    /* ... work ... */
    ret = 0;

cleanup:
    free(b);        // free(NULL) is safe, so no extra checks needed
    free(a);
    return ret;
}
```

10. Related functions

Function	Purpose
`memset(p, 0, n)`	Fill n bytes with a value (zero out a buffer)
`memcpy(dst, src, n)`	Copy n bytes; regions must NOT overlap
`memmove(dst, src, n)`	Copy n bytes; overlap is allowed
`memcmp(a, b, n)`	Compare n bytes
`strdup(s)`	Allocate and copy a string (POSIX; standard in C23). Must be freed
`alloca(n)`	Allocates on the stack (non-standard, avoid)
`VLA: int a[n];`	Variable-length array on the stack (optional in C11+, risk of stack overflow)

11. Finding bugs with tools

AddressSanitizer (fast, built into gcc / clang)

```
gcc -Wall -Wextra -g -fsanitize=address calloc2.c -o calloc2
./calloc2
```

Detects buffer overflows, use after free, double free, and (on Linux) leaks, with a stack trace.

Valgrind

```
gcc -g calloc2.c -o calloc2
valgrind --leak-check=full --show-leak-kinds=all ./calloc2
```

Reports leaks and invalid reads / writes. Look for *'definitely lost'* bytes and *'Invalid write / read'* lines. A clean run ends with: *All heap blocks were freed -- no leaks are possible*.

Useful compiler flags

```
gcc -Wall -Wextra -Wpedantic -Wmisleading-indentation -g
```

12. One-page cheat sheet

Function	Signature	Initial value	On failure	Release with
malloc	malloc(bytes)	Garbage	NULL	free
calloc	calloc(count, size)	Zero	NULL	free
realloc	realloc(ptr, new_bytes)	Old data kept, new part garbage	NULL (old block still valid)	free (the new pointer)
free	free(ptr)	n/a	n/a	n/a

Golden rules

- Every allocation: **check** it, **use** it within bounds, **free** it exactly once.
- Never touch memory after it is freed.
- Never lose the only pointer to a block.
- Pair every early return with proper cleanup.

Quick template

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t n = 4;
    int *arr = calloc(n, sizeof(*arr));
    if (arr == NULL)
    {
        perror("calloc");
        return 1;
    }

    for (size_t i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    arr = NULL;
    return 0;
}
```